

1. My own Javap
2. Invoke the method of the given class

class Class

- It stores the meta data of the classes loaded in the JVM
- The metadata consists of the information about the classes like
 1. super class, super interfaces, fields, methods, ctors, annotations,...

```
Class c1 = Class.forName("java.lang.String") // way-1
```

```
Class c2 = String.class // way-2
```

```
String s1 = "sunbeam";
```

```
Class c3 = s1.getClass(); // way-3
```

My own Javap

- class name
- package name
- super class
- Interfaces
- fields
- ctors
- methods

Annotations

- It is used to perform some operations on the given type
- There are 3 types of annotations
 1. Marker Annotation
 - Override, FunctionalInterface
 2. Single Value Annotation
 - SuppressWarnings(value = "unused")
 - SuppressWarnings(value = {"unused", "rawtypes"})
 2. Multi-Value Annotation
 - @RequestMapping(path="/login", method="GET")

Custom Annotation

MetaAnnotations

- The annotations used over another annotations is called as meta annotations
- types of meta-annotations
 1. Documented
 - To keep the information about the annotation inside the java docs
 2. Retention
 - The annotation details will be available only in
 - a. Source -> details are present in the .java file only
 - b. Class -> details are present in the .class file only
 - c. Runtime -> details are present even at the time of execution

- NativeMethodStack
- native methods
 - JNI

MultiThreading

- Process
- Program in Execution

- Thread
- It is a light weight process

